

04

Módulo 04 - Vision Transformers

Visão de Computador e Stable Diffusion

Resumo do Módulo

- ◆ Visão de computador na era dos LLMs.
- ◆ Vision Transformer.
- ◆ Atenção em Vision Transformers.
- ◆ Vision Large Language Models.

- ◆ Lab

Visão de computador na era dos LLMs

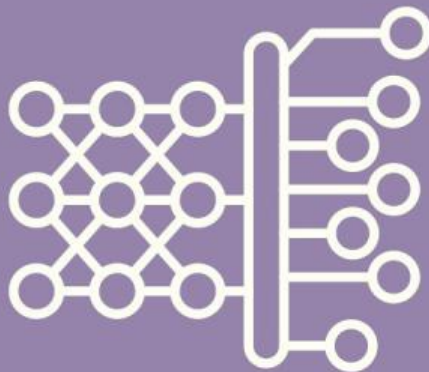
Classical Computer Vision



+Não requer treino

-Menos flexível em cenários complexos

Deep Learning and CNNs



+Alta precisão

- Exige muitos dados e hardware

Multimodal LLM



+Abrange múltiplos domínios

-Necessita treino extenso e variado

AI description

Nenhuma atividade suspeita detetada. O cão da propriedade esteve a brincar na área de vigilância sem demonstrar comportamento de alarme.

10:55

AI description

Veículo particular a manobrar para sair da propriedade. Movimento do carro Toyota Auris (●-●-●) registado na transição para a via pública. Nenhuma atividade suspeita detetada.

12:41

Atenção em Transformers e LLMs

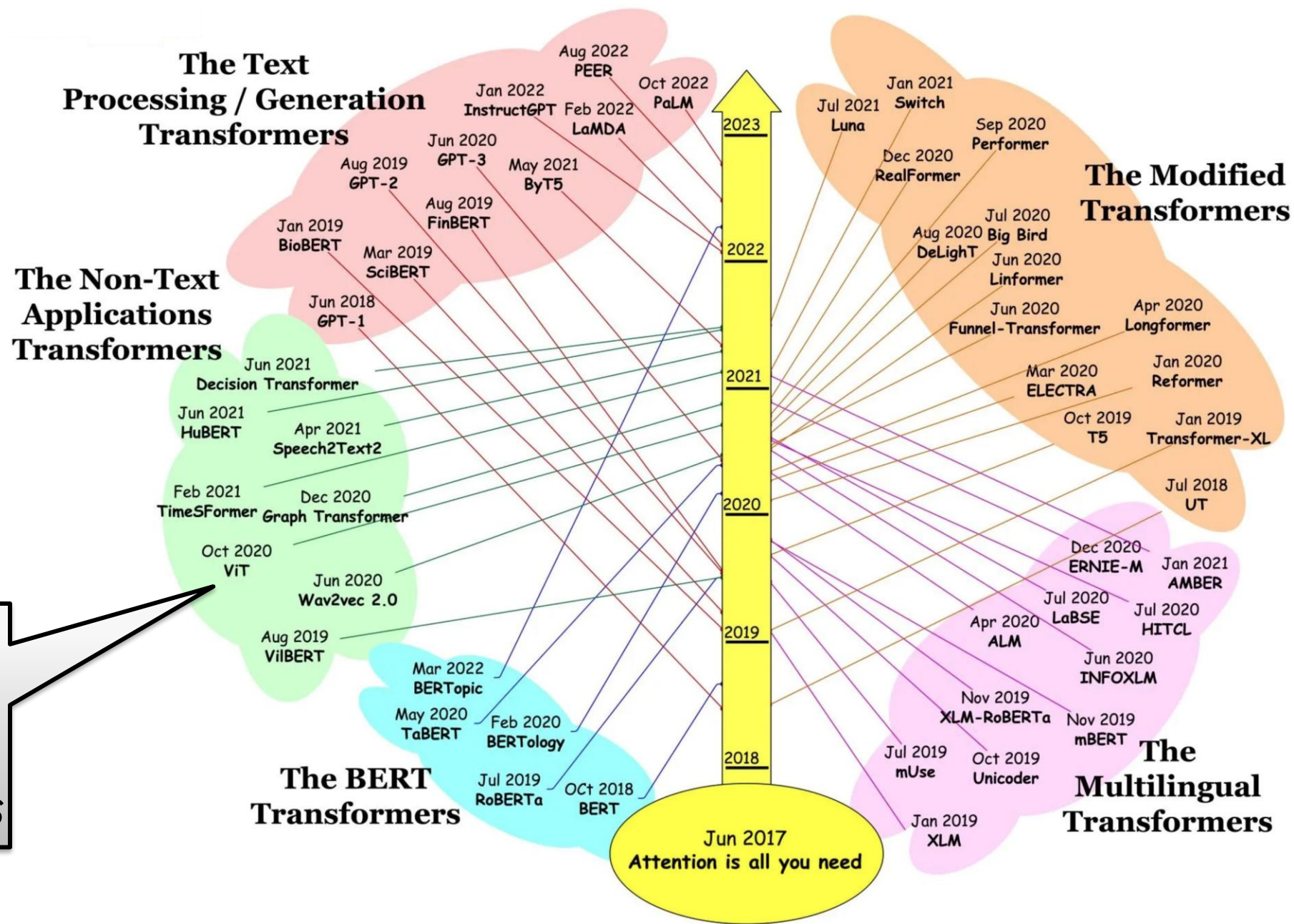
À medida que a frase cresce, a **atenção** desloca-se do primeiro token para os subsequentes.

Mecanismo destacando partes importantes do texto

Pesos de atenção

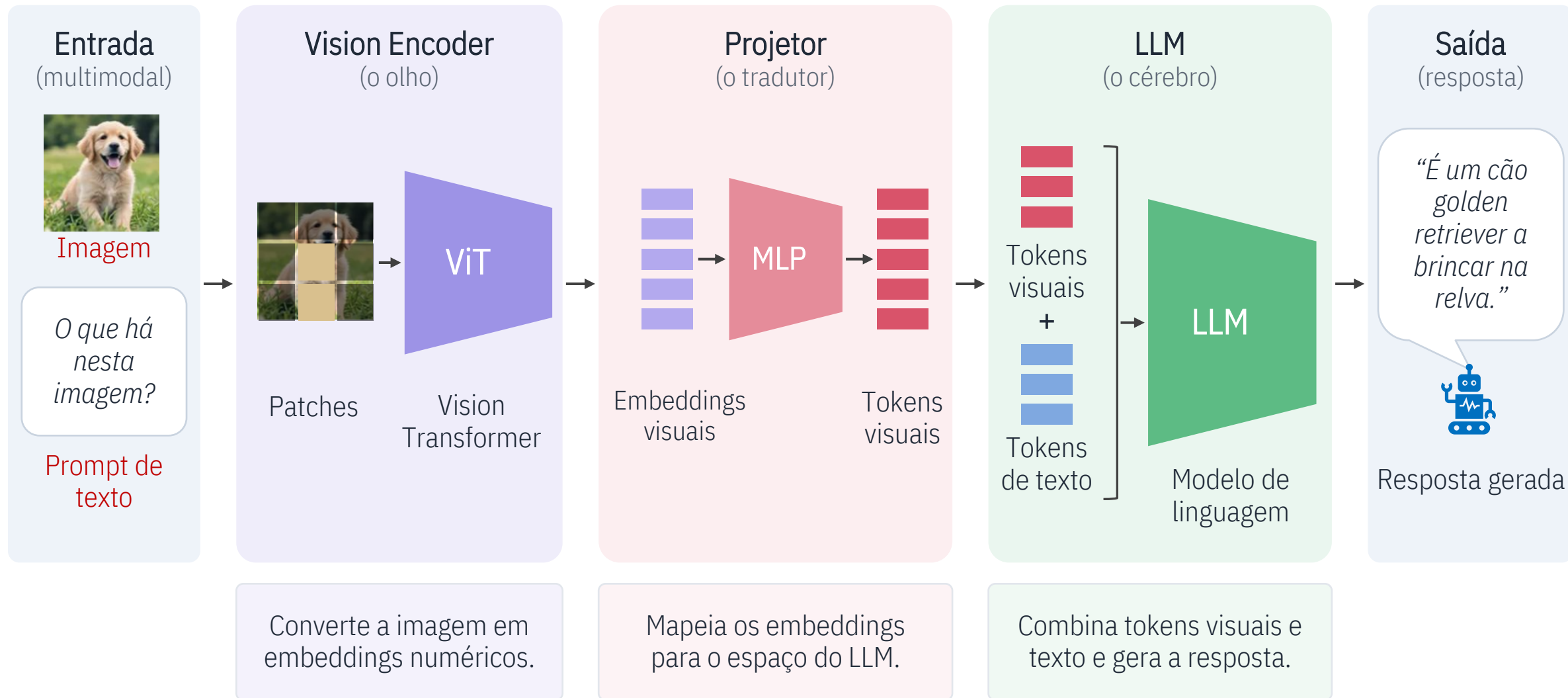
<i>O</i>	[<i>O</i> : 100%]
<i>O gato</i>	[<i>O</i> : 40%, <i>gato</i> : 60%]
<i>O gato dorme</i>	[<i>O</i> : 20%, <i>gato</i> : 40%, <i>dorme</i> : 40%]
<i>O gato dorme no</i>	[<i>O</i> : 10%, <i>gato</i> : 30%, <i>dorme</i> : 30%, <i>no</i> : 30%]
<i>O gato dorme no sofá.</i>	[<i>O</i> : 5%, <i>gato</i> : 25%, <i>dorme</i> : 25%, <i>no</i> : 15%, <i>sofá</i> : 30%]

Vision Transformer



Vision Transformer (ViT) é um excelente exemplo de CNNs usando Transformers

Arquitetura de um Vision LLM

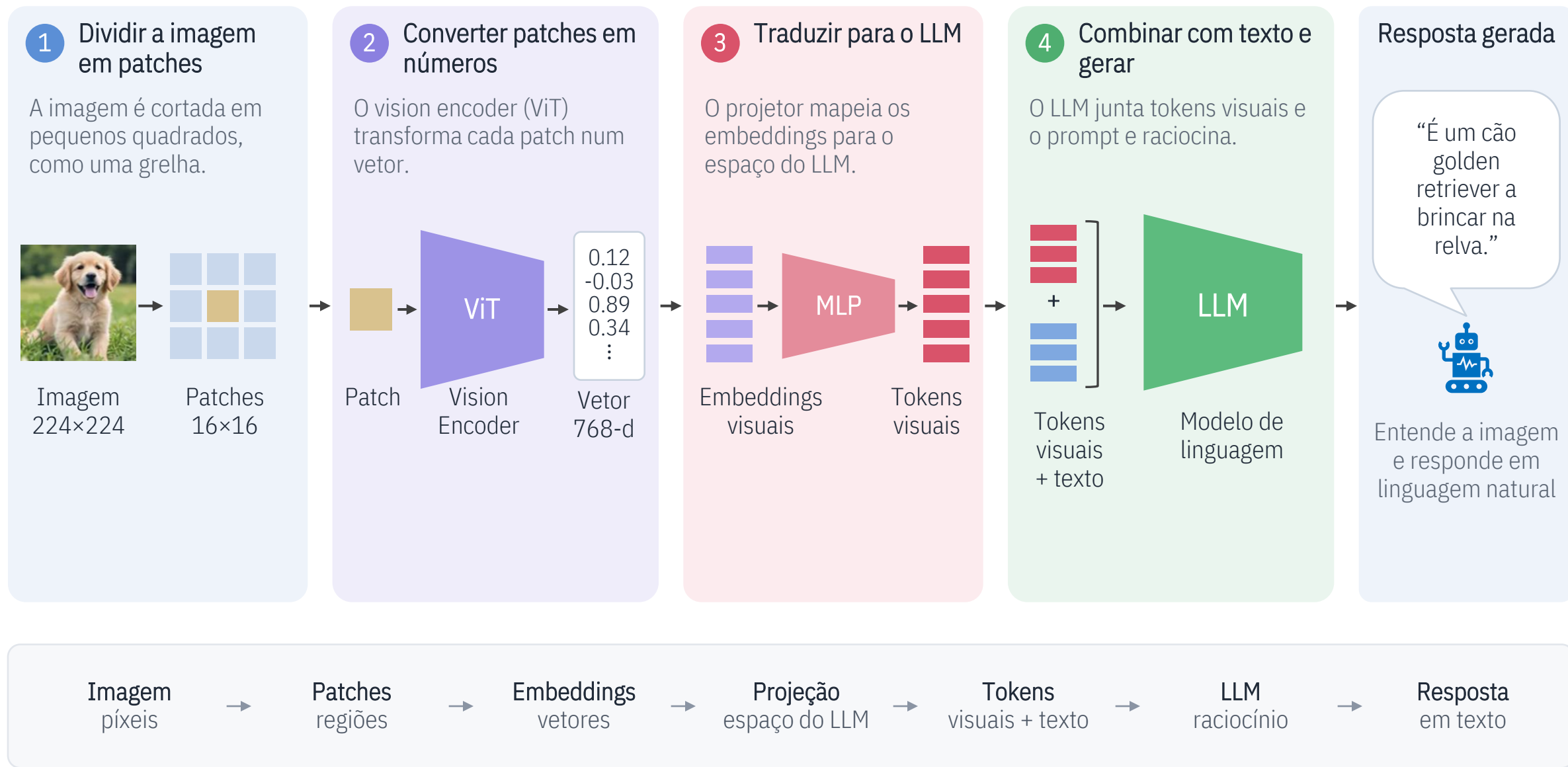


←-----

Compreensão de imagem num espaço de linguagem partilhado

-----→

Dos píxeis às palavras



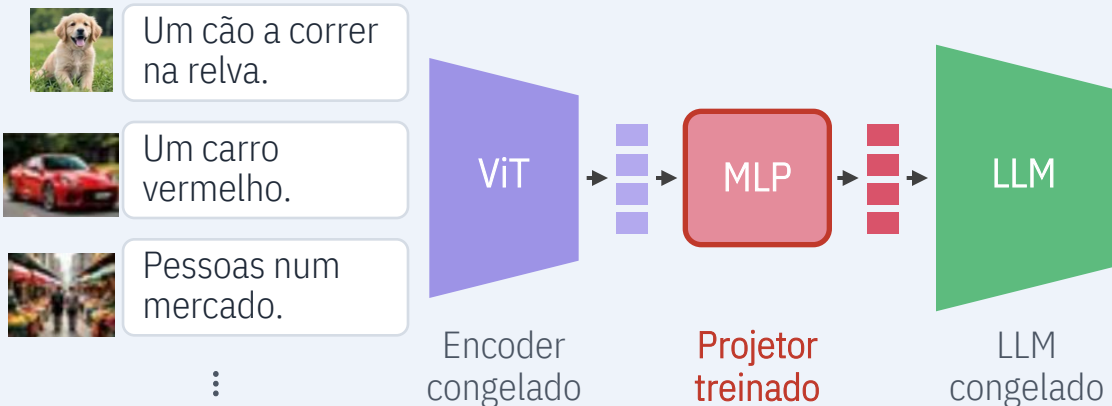
Treino de um Vision LLM em duas fases

1 Fase 1: alinhamento imagem-texto

Ensina o modelo a ligar imagens a linguagem

Pré-treino

Pares imagem + legenda



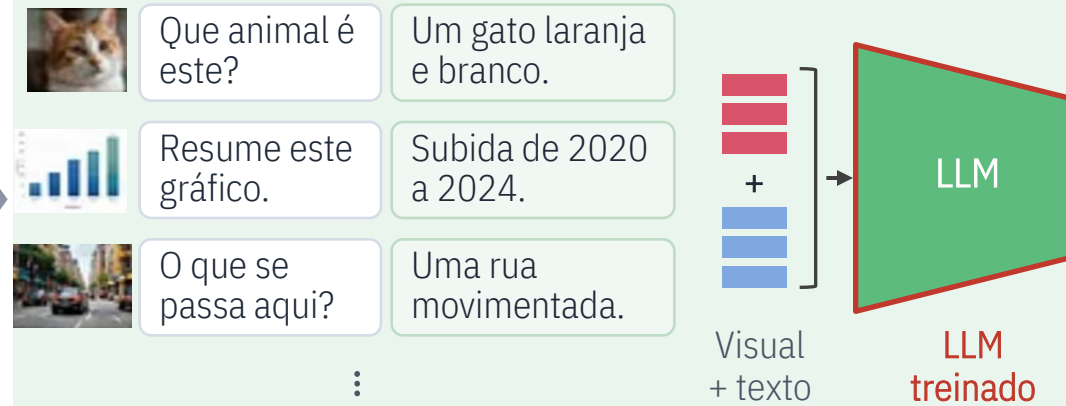
Objetivo: criar um espaço partilhado onde imagens e descrições ficam alinhadas.

2 Fase 2: instruction tuning

Ensina o modelo a seguir instruções sobre imagens

Fine-tuning

Exemplos com instruções



Objetivo: seguir instruções, responder a perguntas e conversar sobre imagens.

Dados de treino

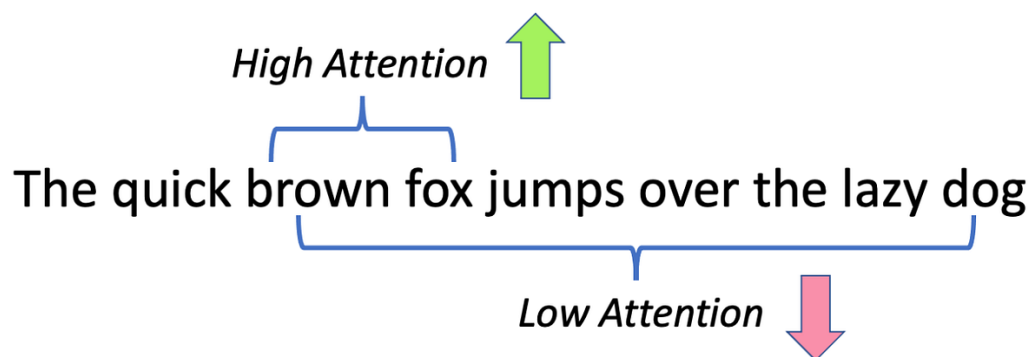
Imagens e legendas da web · datasets de investigação · perguntas e respostas visuais escritas por pessoas · documentos e gráficos

Resultado

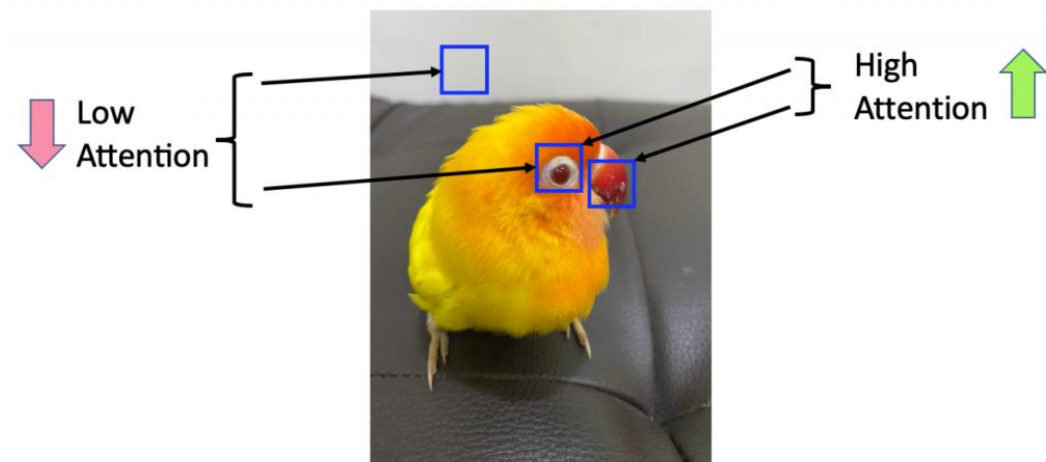
Um modelo que entende imagens e conversa sobre elas em linguagem natural.

Atenção em Vision Transformers

A **Atenção** entre 2 palavras de uma frase é uma medida de **relevância entre elas**.

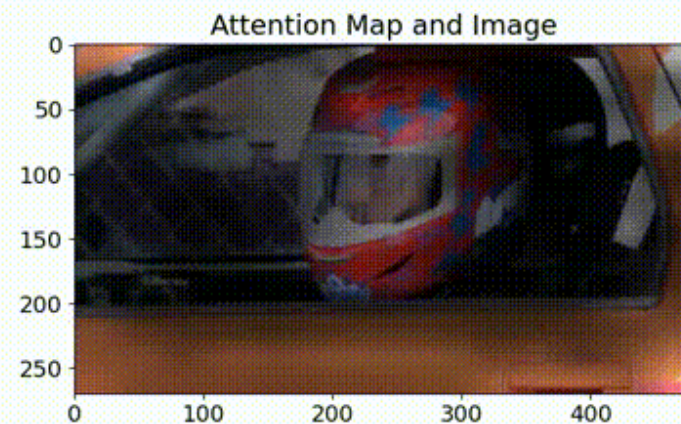
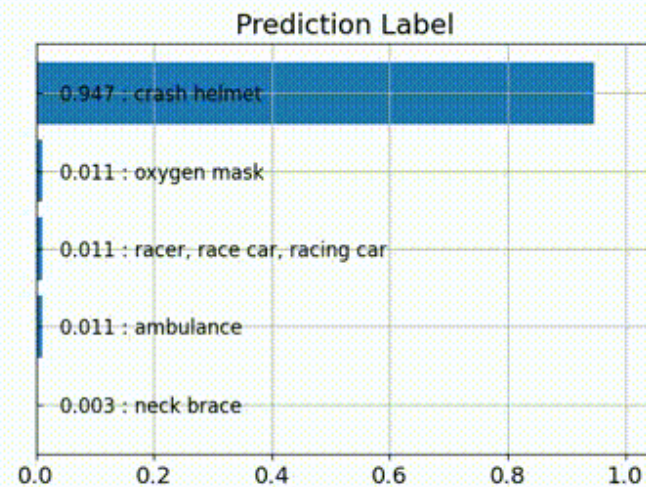
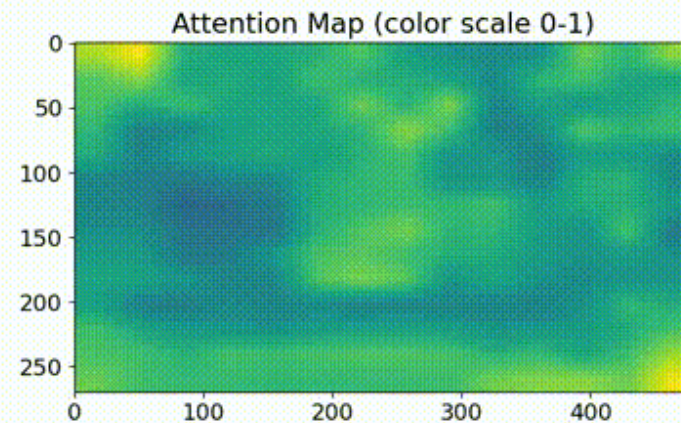
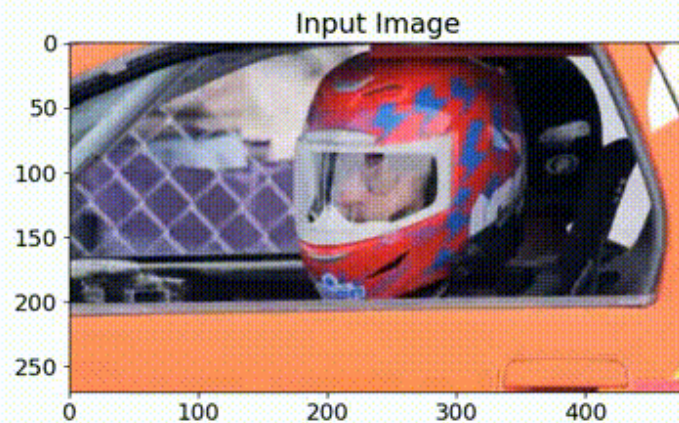


Visual Attention

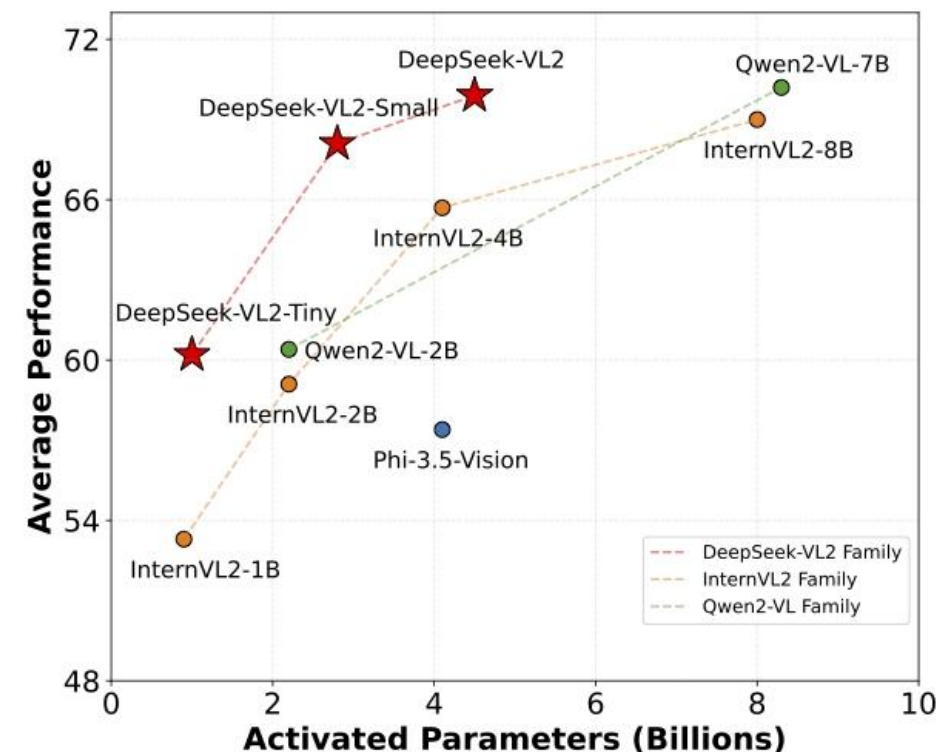
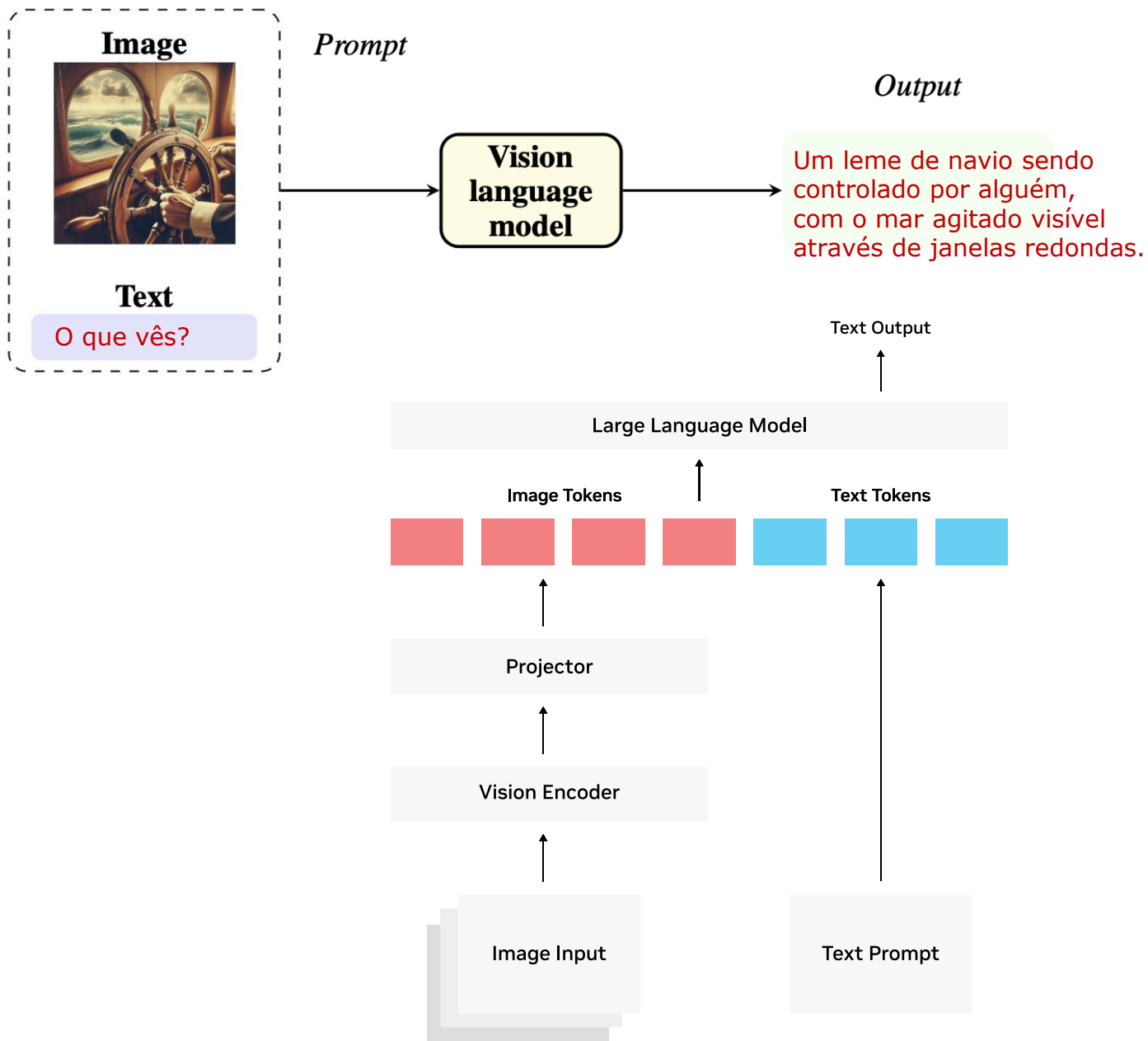


A atenção entre **2 manchas de pássaros é alta**, enquanto a atenção entre qualquer **mancha de parede** e qualquer **mancha de pássaro é baixa**.

Atenção em Vision Transformers

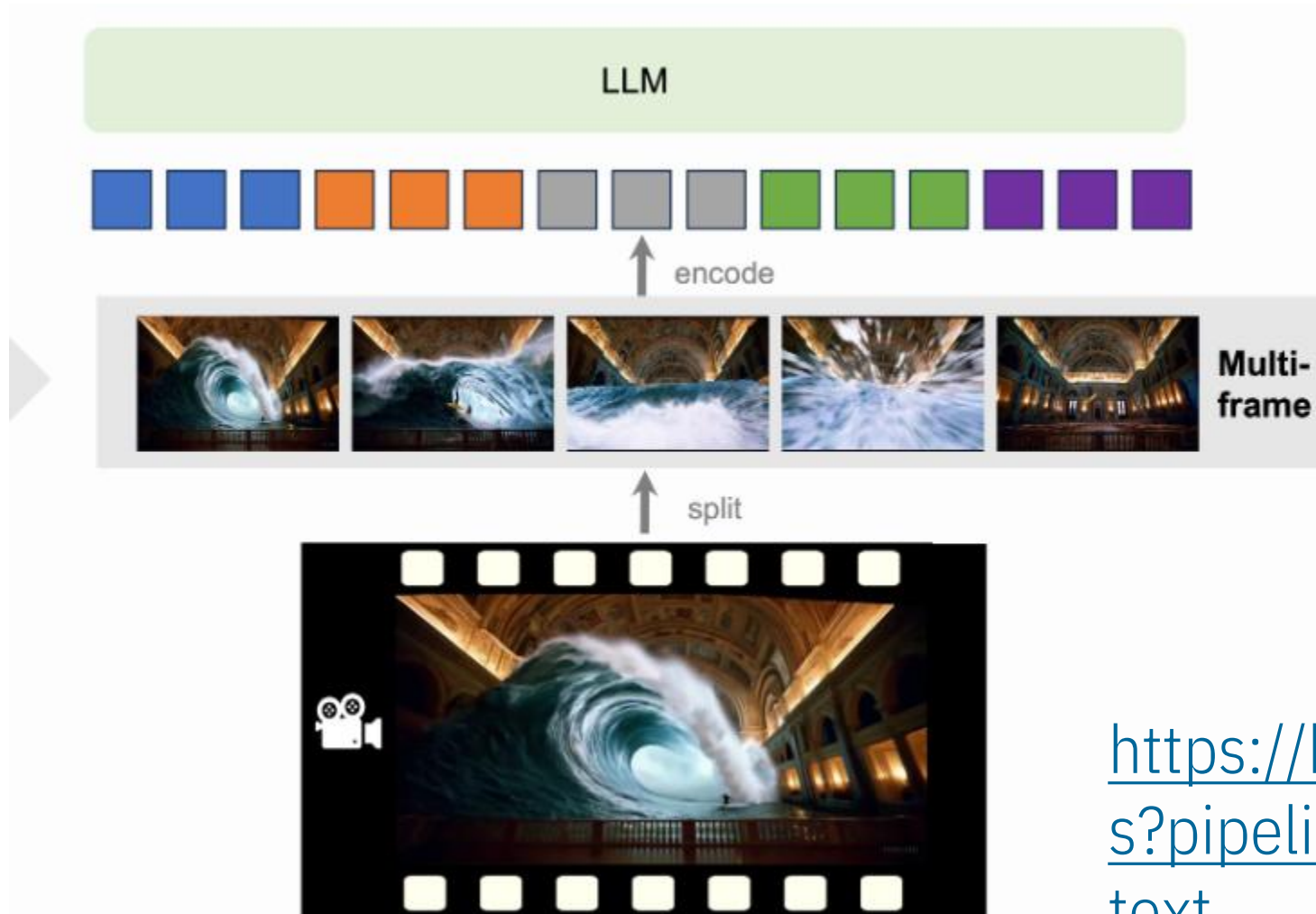


Vision Large Language Model - imagens



https://huggingface.co/models?pipeline_tag=image-text-to-text&sort=trending

Vision Large Language Model - video



https://huggingface.co/models?pipeline_tag=video-text-to-text

A *transformers* é uma biblioteca criada pela Hugging Face que fornece ferramentas para carregar, treinar e utilizar modelos baseados na arquitectura Transformer, de forma simples e consistente

pip install transformers

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from PIL import Image

model_id = "organizacao-ou-utilizador/nome-do-modelo"

model = AutoModelForCausalLM.from_pretrained(
    model_id, trust_remote_code=True, revision=revision
)

tokenizer = AutoTokenizer.from_pretrained(model_id, revision=revision)

image = Image.open('./images/test.jpg')
enc_image = model.encode_image(image)
print(model.answer_question(enc_image, "My prompt", tokenizer))
```

AutoModel é uma fábrica inteligente:

- Lê a configuração do modelo no Hugging Face Hub
- Descobre automaticamente qual a classe correcta a usar
- Faz download dos pesos e cria o objecto do modelo

ForCausalLM indica o tipo de modelo:

- "Causal Language Model" = modelo gerador de texto (tipo GPT)
- Prevê o próximo token, um de cada vez, para gerar respostas.

trust_remote_code=True:

- Alguns modelos têm código personalizado no repositório
- Esta flag permite que esse código seja importado/executado localmente (necessário para métodos extra como `encode_image` e `answer_question`).

AutoTokenizer é outra fábrica automática:

- Vai ao Hub buscar o tokenizador certo para este modelo
- **Tokenizador** = componente que converte texto em números (tokens) que o modelo consegue processar, e depois faz o inverso.

Mesmo que pareça “só texto”, é essencial:

- Sem **tokenizer**, o modelo não percebe a pergunta
- E não consegue decodificar a resposta gerada.

Existem muitas classes na biblioteca mas que continuam estes conceitos

Testar imagens com o 'Qwen/Qwen3-VL-2B-Instruct-GGUF'

1º com o LM Studio

2º com a biblioteca transformers

```
pip install transformers
```

<https://huggingface.co/Qwen/Qwen3-VL-2B-Instruct-GGUF>